```
# First Order Logic using Forward Chaining
# Problem: Prove that Robert is Criminal

facts = {
    ("American", "Robert"),
    ("Enemy", "A"),
    ("Owns", "A", "M1"),
    ("Missile", "M1")
}

# Rules represented as premises and conclusion
rules = [
    # Enemy(A) -> Hostile(A)
    ([("Enemy", "?x")], ("Hostile", "?x")),

    # Missile(x) -> Weapon(x)
    ([("Missile", "?x")], ("Weapon", "?x")),

    # Missile(y) AND Owns(A,y) -> Sells(Robert,y,A)
    ([("Missile", "?y"), ("Owns", "A", "?y")],
     ("Sells", "Robert", "?y", "A")),

    # American(x) AND Weapon(y) AND Sells(x,y,z)
    # AND Hostile(z) -> Criminal(x)
    ([("American", "?x"), ("Weapon", "?y"),
      ("Sells", "?x", "?y", "?z"),
      ("Hostile", "?z")],
     ("Criminal", "?x"))
]


# Forward Chaining Function
def forward_chaining(facts, rules, query):

    inferred = set(facts)

    while True:
        new_facts = set()

        for premises, conclusion in rules:
```

```
def match(index, substitution):
    if index == len(premises):
        return [substitution]

    results = []

    for fact in inferred:
        if fact[0] != premises[index][0]:
            continue

        if len(fact) != len(premises[index]):
            continue

        new_sub = substitution.copy()
        valid = True

        for pattern, value in zip(
            premises[index], fact
        ):
            if pattern.startswith("?"):
                if pattern in new_sub:
                    if new_sub[pattern] != value:
                        valid = False
                        break
                else:
                    new_sub[pattern] = value
            elif pattern != value:
                valid = False
                break

        if valid:
            results.extend(
                match(index + 1, new_sub)
            )

    return results

substitutions = match(0, {})
```

```
            for sub in substitutions:
                result = tuple(
                    sub.get(item, item)
                    if isinstance(item, str)
                    else item
                    for item in conclusion
                )

                if result not in inferred:
                    new_facts.add(result)

        if not new_facts:
            break

        print("\nNew facts derived:")
        for fact in new_facts:
            print(fact)

        inferred.update(new_facts)

    print("\nFinal Result:")

    if query in inferred:
        print("Query is PROVED!")
        print("Robert is a CRIMINAL.")
    else:
        print("Query is NOT PROVED.")


# Query
query = ("Criminal", "Robert")

# Execute
print("Initial Knowledge Base:")
for fact in facts:
    print(fact)

print("\nApplying Forward Chaining...")
forward_chaining(facts, rules, query)
```

OUTPUT:

```
Initial Knowledge Base:
('Enemy', 'A')
('Missile', 'M1')
('American', 'Robert')
('Owns', 'A', 'M1')

Applying Forward Chaining...

New facts derived:
('Weapon', 'M1')
('Hostile', 'A')
('Sells', 'Robert', 'M1', 'A')

New facts derived:
('Criminal', 'Robert')

Final Result:
Query is PROVED!
Robert is a CRIMINAL.
```